Practica 2:

Clases y Ficheros

Metodología de la Programación

1º Ingeniería Informática

Grupo: E

Curso 2025/2026

Índice

1. Objetivos **2**

2. Las clases Pelicula y ConjuntoPeliculas **3**
2.1. Motivación 3
2.2. Variables y Funciones 4

3. Manejo de Ficheros **8**
3.1. Formas de almacenar y recuperar información en ficheros 8
3.1.1. Lectura de ficheros 9
3.1.2. Escritura en ficheros 10
3.2. Funciones save y load de la clase ConjuntoPeliculas 11
3.2.1. Formato del fichero de películas 13

4. Entrega **14**

1. Objetivos

El objetivo de esta práctica es la de practicar el manejo de ficheros en C++. En la programación dirigida a objetos, que es la que estamos usando para las prácticas, la información con la que trabajamos queda almacenada en clases. Sin embargo, una vez se acaba el programa, toda la información almacenada en esas clases se elimina por completo. En esta práctica, vamos a aprender a manejar ficheros lo cual nos permitirá almacenar cualquier tipo de información en ellos lo que, luego, nos permitirá su recuperación y uso en posteriores ejecuciones del programa.

En esta práctica, vamos a trabajar con una base de datos de películas. Para cada película, tenemos almacenados en ficheros su nombre, su año de estreno, su género y su valoración en IMDB en ficheros. La información original proviene del siguiente enlace: `https://www.kaggle.com/datasets/anuselvamathi/imdb-list-of-movies-by-genre`. Para esta práctica, trabajaremos con una versión reducida de la información y un único fichero a fin de que nuestros ordenadores puedan trabajar de forma rápida y eficiente. En prácticas posteriores, utilizaremos memoria dinámica. Los objetivos principales de esta práctica son los siguientes:

- Crear una estructura de datos que nos permita almacenar en memoria y procesar la información contenida en el fichero de datos. Para esto, crearemos las clases Pelicula y ConjuntoPeliculas.
- Crearemos, para las clases Pelicula y ConjuntoPeliculas, varios métodos que nos permitan manejar cómodamente la información almacenada.
- Implementaremos el método leerFichero que nos permitirá leer la información del fichero y almacenarla en la estructura de datos diseñada.
- Implementaremos un método escribeFichero que nos permita almacenar en un fichero toda la información disponible en la estructura de datos diseñada. Es importante que el fichero generado tenga exactamente la misma estructura que el usado para la lectura de forma que el método leerFichero pueda leer todo lo que se genere con escribeFichero.

Para esta práctica utilizaremos memoria estática. Esto conlleva una serie de limitaciones ya que la cantidad de información que puede almacenar la estructura queda reducida drásticamente. En prácticas posteriores, la implementación se modificará de forma que se usará memoria dinámica para el almacenamiento de la información. Es posible, si el alumno lo desea, modificar la implementación proporcionada y entregar directamente esta práctica usando memoria dinámica. Esto puede ahorrar algo de trabajo en el futuro.

El objetivo final del conjunto de prácticas de este curso será la implementación de un sistema de recomendación de películas. Esta práctica implementa la primera parte, la gestión de las películas, siendo objetivos futuros la implementación tanto de los usuarios que las visionan como del propio sistema de

recomendación.

2. Las clases Pelicula y ConjuntoPeliculas

2.1. Motivación

En esta segunda práctica de la asignatura, comenzaremos construyendo una estructura de datos que permita almacenar información sobre una serie de películas. Para ello, construiremos 2 clases diferentes, la clase Pelicula y la clase ConjuntoPeliculas.

La clase Película almacenará la información de una única película. La información que contiene la base de datos para cada película es la siguiente:

- **Id:** Es un número asociado a la película que la identifica unívocamente.
- **Nombre:** Nombre de la película.
- **Año:** Año en que se estrenó.
- **Valoración:** Valoración en la base de datos IMDB.

En la base de datos existe un archivo para cada posible género de la película. En esta práctica trabajaremos sólo con una versión reducida del archivo que contiene las películas de acción. Por tanto, asignaremos ese género a todas las películas que leamos del fichero. Un objeto Película, además de contener variables compatibles que permitan almacenar toda la información, tendrá métodos que permitan la modificación y acceso a toda la información almacenada.

La clase ConjuntoPeliculas contendrá un vector de objetos Película. Su principal función es la de almacenar un conjunto de películas y permitir el acceso a ellas. No estableceremos un orden concreto sobre cómo deben estar almacenadas las películas en el vector aunque sí que implementaremos una función que las ordenará en función de su valoración IMDB. El acceso al vector se hará buscando las películas por su id, por lo que no implementaremos métodos que permitan al usuario de la clase acceder a posiciones concretas del vector. Por el contrario, tendremos métodos que permitan al usuario acceder a la información de las películas o modificarlas en función del id. Por ello, es muy importante evitar que en el objeto ConjuntoPeliculas con el que trabajemos haya 2 películas que tengan el mismo id ya que eso podría acabar dando problemas.

En cuanto al vector utilizado para almacenar los objetos Película, deberemos de tener en cuenta lo siguiente:

- **Memoria estática**: En esta primera versión del programa utilizaremos memoria estática. El uso de memoria estática conlleva las siguientes desventajas:

  - Capacidad limitada de almacenamiento. La cantidad de información que se puede almacenar en memoria estática es mucho menor de la que se puede almacenar si usamos memoria dinámica. Por tanto, para grandes cantidades de información, es mucho mejor utilizar memoria dinámica.

  - Debemos de especificar la cantidad de memoria a reservar antes de la ejecución del programa. Con memoria estática, debemos indicar cuanto espacio queremos reservar en memoria y mantenerla reservada todo el programa. La principal desventaja de esto es que durante todo el programa estamos malgastando recursos de memoria ya que, como no sabemos a priori cuanta memoria vamos a usar, esto nos obliga a reservar grandes cantidades de memoria que puede que luego ni utilicemos. Por el contrario, la memoria dinámica permite utilizar los recursos de memoria en puntos concretos de la ejecución del programa y, al no tener que especificar antes de la ejecución la cantidad de memoria, podemos ajustar y utilizar únicamente la cantidad de memoria que necesitemos en cada momento.

  Si el alumno lo desea, puede modificar la implementación actual para adaptarla al uso de memoria dinámica. Dicho proceso, de todas formas, será llevado a cabo por todos los alumnos en prácticas posteriores.

- **Existe un máximo de tamaño**: Al usar memoria estática, definimos una variable que indica el tamaño reservado que tenemos en memoria para el vector de Películas. Este valor es fijo y no puede ser modificado durante la ejecución del programa. En la versión con memoria dinámica del programa mantendremos esta variable ya que el proceso de reserva es costoso y no debemos hacerlo cada vez que queremos meter un nuevo valor en el vector. Sin embargo, en memoria dinámica, este valor será menor y variable ya que podrá ir aumentando conforme vayamos necesitando más espacio en el vector.

- **Diferencia entre máximo de tamaño y cantidad de elementos en el vector**: El valor que indica el tamaño de memoria reservado difiere de la cantidad de películas que estamos almacenando en el vector en un momento concreto. Por ello, hemos de definir 2 variables de tamaño, una que diga cuánto espacio tenemos reservado, y otra que indique la cantidad de películas almacenadas en el vector. Comentaremos el nombre de todas estas variables en la siguiente sección.

2.2. Variables y Funciones

En esta sección daremos una breve explicación de todas las funciones y variables que deben implementar ambas clases. Se entregará al alumno un proyecto de Netbeans con todos los métodos de las clases vacíos y el objetivo de la práctica será el de implementarlos. Si el alumno utiliza otro IDE, debe sacar de ahí los ficheros .h y .cpp y añadirlos al entorno de progrmaación que utilice. El main.cpp tiene algunas funciones que pueden servir para probar el funcionamiento de dichos métodos. Se recomienda que, aunque

el alumno puede incluir las suyas propias, se prueben también las incluidas. En los ficheros .h del proyecto entregado se explica detalladamente, mediante directrices doxygen, qué debe de hacer cada uno de los métodos. Aun así, incluimos a continuación una breve descripción de qué métodos hay incluidos y qué debe de hacer cada uno.

En cuanto a la **clase Pelicula**, tenemos las siguientes variables y funciones.
Las variables de la clase son las siguientes:

- **int id**: Id de la película. Debe de ser único ya que identifica unívocamente a una película concreta.
- **string nombre**: String que contiene el nombre de la película.
- **float valoracion**: Número flotante que representa la puntuación IMDB obtenida.
- **string genero**: Género de la película. Para esta práctica el género de todas las películas es .ªcción".
- **int anio**: Año de estreno de la película.

Los métodos de la clase son los siguientes:

- **Pelicula()**: Inicia los valores de la película a valores no válidos pero identificables. Este constructor se utilizará sobre todo para poder crear un vector de Peliculas y poder trabajar con él. Recordemos que para poder crear un vector de objetos es necesario que el objeto utilizado en el vector deba tener implementado el constructor por defecto.
- **Pelicula(int newid, string newnombre, int newanio, float newvaloracion, string newgenero)**: Crea un nuevo objeto película inicializando todos los valores de las variables que lo componen. newid es id de la película. Debe de ser un valor único para cada película. newnombre es el nombre de la película, newanio es el año de estreno de la película. newvaloracion es la valoración en la base de datos IMDB de la película. Finalmente, newgenero identifica el género de la película.
- **Pelicula(const Pelicula & orig)**: Constructor de copia. Como no hay variables con memoria dinámica no hace falta implementarlo.
- **string to_string():** Genera un string que contiene todos los valores de las variables del objeto Película separados por espacios. El objetivo de este método es poder generar un string que poder mostrar por pantalla. Este método devuelve un string que contiene los siguientes datos mostrados en este orden: ïd nombre género año valoración".
- **~Pelicula()**: Destructor del objeto película. Como no hay ahora mismo datos que usen memoria dinámica no hay que implementarlo. En memoria dinámica usa delete para desasignar la memoria utilizada.

- **int & setId():** Modifica el id de la película. Devuelve una referencia al entero id que permite modificar su valor.
- **string & setNombre()**: Modifica el nombre de la película. Devuelve una referencia al string nombre que permite modificar su valor.
- **float & setValoracion()**: Modifica la valoración IMDB de la película. Devuelve una referencia al flotante valoracion que permite que su valor sea modificado.
- **string & setGenero()**: Modifica el género asociado a la película. Devuelve una referencia al string genero que permite modificar su valor.
- **int & setAnio()**: Modifica el año de estreno de la película. Devuelve una referencia al valor de año que permite modificar su valor.
- **int getId()**: Devuelve un valor entero que contiene el id de la película.
- **string getNombre()**: Devuelve un string que contiene el nombre de la película.
- **string getGenero():** Devuelve un string que contiene el género de la película.
- **int getAnio()**: Devuelve un entero que contiene el año en que se estrenó la película.
- **float getValoracion()**: Devuelve un valor flotante en el intervalo [0,10] que contiene la valoración en la base de datos IMDB de la película.

En cuanto a la clase **ConjuntoPeliculas**, pasamos a listar sus variables y funciones.

Las variables que contiene la clase son:

- **static const int reservado**: Cantidad de memoria reservada. Indica el número máximo de objetos que puede almacenar el vector. Como el valor no puede ser modificado durante la ejecución, debe de ser static const. Cuando pasemos la clase a memoria dinámica será sustituido por int reservado.
- **Pelicula conj[reservado]**: Vector estático de objetos Pelicula que almacena un conjunto de películas. Como máximo se pueden almacenar la cantidad de películas que indica la variable reservado. En el caso de memoria dinámica lo sustituiremos por Pelicula * conj.
- **int numpeliculas**: Indica el número de películas que tiene el objeto almacenado actualmente. Las películas deben de almacenarse de forma continua en el vector, quedando todos los espacios libres en las posiciones finales del vector.

Los métodos que contiene la clase son:

- **ConjuntoPeliculas():** Constructor por defecto de la clase. Genera un vector vacío, sin ninguna película almacenada.

- **ConjuntoPeliculas(const ConjuntoPeliculas & orig)**: Constructor de copia. Si el vector es estático, no hace falta implementarla.

- **void leerFichero(string rutaFichero, int numdatos)**: Este método lee numdatos películas y las almacena en el vector de películas. Si el vector contuviera alguna película previamente, entonces lo vaciamos. En posteriores versiones no estáticas de este método quitaremos la variable numdatos permitiendo que se agreguen todas las películas que haya almacenadas en un fichero. Los parámetros de este método se muestran a continuación:

  - rutaFichero: Ruta del fichero que contiene la información de las películas a almacenar en el objeto.

  - numdatos: Número de películas que vamos a leer de ese fichero (siempre leeremos las numdatos primeras películas almacenadas). En el caso de que numdatos sea mayor que el número de elementos del fichero entonces leemos sólo la información disponible. Por ello, debemos llevar un control en este método de cuando se ha terminado de leer toda la información.

- **~ConjuntoPeliculas()**: Destructor de la clase ConjuntoPeliculas. Sólo es necesario implementarlo si se usa memoria dinámica.

- **void aniadePelicula(int newid, string newnombre, int newanio, float newvaloracion,string newgenero)**: Añade una nueva película al vector de películas. La película se añade en la siguiente posición libre del vector. El método tiene como entradas todas las variables miembro del objeto Pelicula que definirán el nuevo objeto.

- **string to_string():** Genera un string que contiene toda la información de todas las películas contenidas en el vector. El objetivo de este método es poder generar un string que poder mostrar por pantalla. Este método devuelve un string en donde se muestra una película en cada línea. (Las películas se separan por el carácter \n dentro del string definido). Puede usarse el método to_string de la clase Película dentro de este método.

- **void escribeFichero(string rutaFichero)**: Almacena las películas que hay disponibles en el vector en un fichero csv. El fichero csv generado debe seguir exactamente el mismo formato que el que tiene el fichero de entrada utilizado en la práctica. Por tanto, el fichero que genera el método escribeFichero debe de ser capaz de ser leído por el método leerFichero. Este método sólo tiene un parámetro, rutaFichero, que indica la ruta en donde queremos que se almacene el fichero generado.

- **void ordenaporranking()**: Ordena las películas del vector conj de mayor a menor puntuación IMDB,

es decir, utilizando la variable valoración de cada una de las películas. Este método funciona sobre el propio vector del objeto con lo cual no se genera ningún vector aparte y el orden previo que hubiera se pierde.

- **void borrar(int id)**: Elimina una única película del vector de películas. Concretamente la que tenga el id indicado como parámetro. Si no hubiera ninguna película con el id introducido, entonces el método no hace nada.

- **Pelicula at(int id)**: Devuelve un objeto película del vector de películas. Concretamente, aquella que tenga como id la indicada como parámetro. El parámetro de este método es el id de la película a obtener. El método devuelve el objeto película que tenga esa id. Si no existiera, se devuelve una película vacía.

- **void clear():** Elimina toda la información del vector. Dependiendo de si el vector es estático o dinámico la implementación varía.

El alumno tiene libertad de incluir en las clases todas las funciones que crea necesarias para la correcta implementación de los métodos de las clases propuestos.

3. Manejo de Ficheros

3.1. Formas de almacenar y recuperar información en ficheros

El lenguaje de programación C++ incluye una serie de funciones que nos permiten trabajar con archivos. Antes de nada, a la hora de trabajar con archivos, tenemos que tener en cuenta lo siguiente:

- Es importante saber en todo momento donde se encuentran los archivos en los que estamos volcando/leyendo información.

- A la hora de leer un archivo, debemos asegurarnos de que el archivo realmente existe. En caso de no ser así, el código dará muchos errores. Existen funciones que permiten saber si se ha accedido adecuadamente al archivo. Es muy importante usarlas siempre.

- De cara a la asignatura, debemos guardar los ficheros generados asociados al proyecto en una carpeta que esté dentro del proyecto. Para ello, podemos crear un directorio data en la carpeta del proyecto que contenga los archivos.

Para poder acceder a la funcionalidad de lectura y escritura de ficheros, debemos incluir la librería <fstream>. Esta biblioteca contiene todas las funciones de las que estamos hablando. Aun así, para leer y trabajar con strings a nivel de ficheros, es posible que tengamos también que incluir la librería <string>. A continuación, proporcionaremos nociones básicas sobre la lectura y escritura de ficheros. Esta información será complementada y ampliada en teoría.

3.1.1. Lectura de ficheros

Las funciones y objetos que están relacionadas con la lectura de ficheros son las siguientes:

- **Clase ifstream**: Es la clase que nos permite obtener información de un fichero. Antes de nada, lo primero es declarar un objeto de esta clase.
- **Método open(string ruta)**: Permite abrir el fichero que está localizado en la ruta indicada en el string ruta. Antes de leer, debemos asociar el objeto de clase ifstream creado a un fichero concreto del sistema. Esta asociación se hace mediante el método open.
- **Método is_open()**: Si devuelve true significa que el fichero se ha abierto correctamente. En caso de devolver false, suele deberse a que no hemos ejecutado el método open o a que la ruta que hemos indicado para el fichero no existe.
- **Operador »**: Funciona de forma similar a como lo usábamos con cin sólo que esta vez lo aplicamos sobre el objeto ifstream. Su función es la de leer el dato apuntado y pasar a apuntar al siguiente. Es importante que el dato leído sea compatible con la variable sobre la que se aplica el operador.
- **Función getline(ifstream fi,string cad,char delimitador)**: Lee todo lo que hay en el fichero hasta llegar al carácter delimitador. Si queremos leer una línea completa podemos utilizar el delimitador '\n'. Existen otros posibles delimitadores como el tabulador ' \t' o el carácter punto y coma ';'. Cualquier carácter que incluyamos en delimitador será usado con ese fin. El resultado se almacena siempre en un string por lo que para convertirlo a otros tipos de dato debemos utilizar las siguientes funciones:
  - *int stoi (string)*: Convierte un string en entero.
  - *float stof (string)*: Convierte un string en un float.
- **Método close()**: Es muy importante que, una vez que hemos terminado de trabajar con el fichero, cerremos la conexión de la clase con el fichero. Esto evitará posibles problemas con el fichero.

Cuando abrimos un fichero, el sistema crea un puntero que apunta al principio de éste. Cada vez que leemos un dato utilizando el operador » o la función getline, el puntero se mueve hacia adelante apuntando a aquellos datos que aún no se han leído. Por lo general, en aras de la eficiencia, debemos programar nuestras funciones de lectura y escritura de ficheros de forma que todos los datos se lean uno tras otro hasta llegar al final del fichero. Para saber si hemos llegado al final del fichero, podemos utilizar la función eof() que devuelve true si ya no quedan más datos por leer del fichero y false si el puntero aún no se ha llegado al final.

En la Figura 1, podemos ver un ejemplo de un código que lee 3 datos de un fichero. Los datos pueden estar separados por espacios dentro del fichero o en 3 líneas diferentes. Por último, en la Figura 3,

```
string a;
int x;
char y;

int main ()
{
ifstream mi_fichero_entrada; //  flujo de entrada

mi_fichero_entrada.open("datosEntrada.txt");

...
if (mi_fichero_entrada.is_open()){
   mi_fichero_entrada >> a; // lee hasta encontrar separador
   mi_fichero_entrada >> x >> y ; // lee entero  char
}

mi_fichero_entrada.close();
```

Figura 1: Lectura de datos de fichero utilizando ».

podemos ver un ejemplo de lectura de ficheros utilizando la función getline que nos proporciona el include string que permite leer un fichero línea a línea:
Este código mostrado en la Figura muestra como leer las primeras 50 líneas de un fichero. Puede usarse la función getline como delimitador del bucle de lectura. Si la función getline devuelve false, entonces significa que se ha llegado al final del fichero y no se ha podido leer nada más.

3.1.2. Escritura en ficheros

La escritura en ficheros puede realizarse de forma cómoda utilizando la clase ofstream y el operador « que, como ya vimos cuando usamos cout, se puede utilizar con todos los tipos de datos base. Los objetos y métodos que hay que tener en cuenta en este caso son las siguientes:

- **Clase ofstream**: Es la clase que permite abrir un fichero para su escritura. Antes de nada, lo primero es declarar un objeto de esta clase.

- **Método open(string ruta)**: Permite abrir el fichero que está localizado en la ruta indicada en el string ruta para su escritura. Antes de escribir nada, hay que ejecutar este método para poder asociar el objeto fo creado a un fichero concreto del sistema. Esta asociación se hace mediante el método open. El método open tiene un parámetro que permite añadir información al fichero sin borrar lo que ya esté escrito en él. Por defecto, sino se indica nada, el fichero es sobreescrito.

- **Método is_open()**: Si devuelve true significa que el fichero se ha abierto correctamente. En caso de devolver false, suele deberse a que no hemos ejecutado el método open o a que la ruta que hemos

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main(){
    /* Leer las 50 primeras líneas de un fichero y/o parar cuando acabe el fichero*/
    ifstream fi;
    string * cadenas;
    bool findefichero = false;
    cadenas = new string [50];
    int numlineas=0;

    fi.open("textolargo.txt");
    if(fi.is_open()){
        while(getline(fi,cadenas[numlineas])
              && numlineas < 50){
            numlineas++;
        }
    }
    fi.close();
    cout << "Numero de lineas:" << numlineas<< endl;
    for(int i = 0; i < numlineas; i++){
        cout << cadenas[i] << endl;
    }
}
```

```
E:\pruebacodeblock\pruebaficheroscodigos\lecturayescrituralineas.exe
Numero de lineas:14
En lo que concierne al sistema operativo un archivo es, en la mayor├¡a de los casos,
simplemente un flujo unidimensional de bytes, que es tratado por el sistema operativo
como una ├│nica unidad l├│gica. Un archivo de datos inform├ítico normalmente tiene un tama├▒o,
que generalmente se expresa en bytes; en todos los sistemas operativos modernos, el tama├▒o puede
ser cualquier n├│mero entero no negativo de bytes hasta un m├íximo dependiente del sistema. Depende
del software que se ejecuta en la computadora el interpretar esta estructura b├ísica como por ejemplo
 un programa, un texto o una imagen, bas├índose en su nombre y contenido. Los tipos especiales de
archivos, como los nodos de dispositivo que representan simb├│licamente partes del hardware, no
consisten en un flujo de bytes y no tienen tama├▒o de archivo.
Los datos de un archivo inform├ítico normalmente consisten en paquetes m├ís peque├▒os
de datos (a menudo llamados registros o l├¡neas) que son individualmente diferentes
pero que comparten alg├│n rasgo en com├│n. Por ejemplo, un archivo de n├│minas puede
contener datos sobre todos los empleados de una empresa y los detalles de su n├│mina;
cada registro del archivo de n├│minas se refiere ├│nicamente a un empleado, y todos loa

Process returned 0 (0x0)   execution time : 1.504 s
Press any key to continue.
```

Figura 2: Lectura de datos de fichero utilizando la función getline.

indicado para el fichero no es accesible.

- **Operador «** : Funciona de forma similar a como lo hemos estado usando con cout. En vez de poner cout indicamos el objeto de la clase ofstream que hemos creado. Este operador permite trabajar con todo tipo de variables simples. Para su uso con objetos de clases debemos sobreescribir el operador « en la clase cosa que aprenderemos próximamente.

- **Método close()**: Es muy importante que, una vez que hemos terminado de trabajar con el fichero, cerremos la conexión de la clase con el fichero. Esto evitará posibles problemas con el fichero. Si no se hace, parte de la información que se envíe para escritura podría no terminar por escribirse en el fichero.

Para terminar, veamos un ejemplo de código en el que leemos un fichero y luego creamos un fichero con la misma información.

3.2. Funciones save y load de la clase ConjuntoPeliculas

La clase ConjuntoPeliculas tiene dos métodos, leerFichero y escribeFichero, que permiten cargar en la estructura de datos la información almacenada en un fichero y viceversa. La función escribeFichero almacenará la información contenida en la estructura de datos en un fichero. Este fichero debe tener una estructura específica ya que la función leerFichero debe de ser capaz de leerlo y cargar la información leída en un objeto de la clase ConjuntoPeliculas en una ejecución posterior (o en la misma). De esta manera, seremos capaces de almacenar la información de la estructura de datos de películas en un fichero y, poste-

```
#include <cstdlib>
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char** argv) {
    ifstream fi;
    ofstream fo;
    string cadena;
    float numero;
    int entero;

    fi.open("./data/entrada.txt");
    if(fi.is_open())
    fi >> cadena >> numero >> entero;
    fi.close();

    fo.open("./data/salida.txt");
    if(fo.is_open())
    fo << cadena << " " << numero << " " << entero << endl;
    fo.close();
}
```

Figura 3: Lectura y escritura de datos con los operadores « y ».

riormente, podremos cargar el fichero y recuperar la información almacenada. Puede verse la estructura que tiene que tener el fichero en el fichero dado como ejemplo para la lectura.

Es muy importante, de cara a la organización del proyecto informático, que los ficheros generados se generen todos en un único lugar. Para ello, crearemos un directorio data dentro de la carpeta del proyecto en donde guardaremos todos los ficheros que genere nuestro programa. Por tanto, al finalizar la práctica, nuestro proyecto incluirá, entre otros, los siguientes directorios:

- dist: Almacena los ejecutables del proyecto.
- include: Almacena los ficheros .h.
- src: Almacena los ficheros .cpp.

- data: Almacena los ficheros de texto generados por el proyecto.

3.2.1. Formato del fichero de películas

En esta sección, describimos brevemente el formato que debe de tener el fichero que debemos leer y escribir con la información de las películas. Los ficheros con los que vamos a trabajar en esta práctica son ficheros .csv. Su principal función es la de almacenar información que proviene de tablas. Todos los ficheros csv tienen el formato que describimos a continuación:

- Cada item viene representado en una línea del fichero.
- De cada item, se almacenan una serie de características. Los valores de dichas características se almacenan en cada línea del fichero separados por el carácter ;.
- La definición del significado de cada una de las características del item almacenado puede venir indicado en la primera línea del fichero en donde, separados por ;, se indica el significado de cada una de las características del item. A esta línea la denominamos cabecera.

Concretamente, en esta práctica, vamos a trabajar con ficheros csv que almacenan información de películas. En cada línea se encuentran los datos de cada una de las películas almacenadas. Y, separados por el carácter ;, encontramos cada uno de los datos asociados a cada película. El fichero proporcionado en la práctica tiene la siguiente estructura:

- **Cabecera del fichero**: Breve descripción de cada uno de los campos separados por ;.
- A partir de la segunda línea se listan las películas.
- Para cada película se lista, en este orden, su id, nombre, año y valoración IMDB, todo separado por el caracter ;.
- La valoración viene dada en un número flotante donde usamos , para los números decimales.

Finalmente, para que quede claro el formato que se pide, mostramos un ejemplo en la Figura 4.

Un detalle importante con el que tendréis que tratar a la hora de leer el fichero es que la función stof que transforma un string en un valor flotante siempre trabaja con . y no con , en los números decimales. Por tanto, es necesario trabajar con las funciones de la clase string, o encontrar otra solución, para que la transformación de string a flotante y viceversa se haga correctamente.

```
;Movie title;Released Year;ImDb Rating
0;Thor: Love and Thunder;2022;6,4
1;Top Gun: Maverick;2022;8,4
2;Brahmastra Part One: Shiva;2022;5,6
3;The Woman King;2022;6,1
4;End of the Road;2022;4,7
5;Bullet Train;2022;7,5
6;Moonfall;2022;5,1
7;Everything Everywhere All at Once;2022;8,1
8;Morbius;2022;5,2
9;The Lord of the Rings: The Fellowship of the Ring;2001;8,8
10;Top Gun;1986;6,9
11;Samaritan;2022;5,7
12;Prey;2022;7,2
13;Jurassic World Dominion;2022;5,7
14;The Karate Kid Part III;1989;5,3
15;The Batman;2022;7,9
16;The Gray Man;2022;6,5
17;Spider-Man: No Way Home;2021;8,3
18;The Northman;2022;7,1
```

Figura 4: Formato del fichero de películas.

4. Entrega

La entrega consistirá en un fichero comprimido que contendrá la carpeta del proyecto con las clases Pelicula y ConjuntoPeliculas implementadas. En el fichero main de dicho proyecto se han incluido varias funciones de prueba que podéis utilizar para comprobar el correcto funcionamiento de las clases. Se recomienda ir descomentándolas una a una conforme se vayan implementando las funciones para comprobar su correcto funcionamiento. Las funciones que tengan más líneas de código deben de ir comentadas.

La fecha de entrega de la práctica será la indicada en PRADO.

Se recuerda que es muy importante que el proyecto **compile perfectamente** ya que, de no hacerlo, la entrega no será válida.